Data Analysis in Geophysics
Machine Learning Classification

Diego Andres Castro Rojas

1 Machine Learning and Neural Net

In this introductory machine-learning exercise I classified objects in images, this images are handwritten numbers and train an ANN to correctly classify each single digit number. To accomplish this task I use a Multi-Layer Perceptron algorithm implemented in Python's scikit-learn module AstroDave and Cukierski 2012. For more details and tutorials on scikit-learn see following link.

All the code develop for this module can be find in: GitHub Data_Analysis_UofM

1.1 Neural Nets and the Kaggle handwritten number data

We will use a designated training data set from kaggel data, each row in the data file consists of a class label (integer between 0 to 9) and a 28*28=784 element vector specifying a gray scale image (size=nx*ny=28*28) of a handwritten number (thus total length of each row = 784). Gray scale images (unlike RGB images) just specify the shade of gray with integers between 0 to 255 for each pixel that is store in a NumPy zipped file (mnist_scaled.npz) a random sample of this data set is shown bellow:

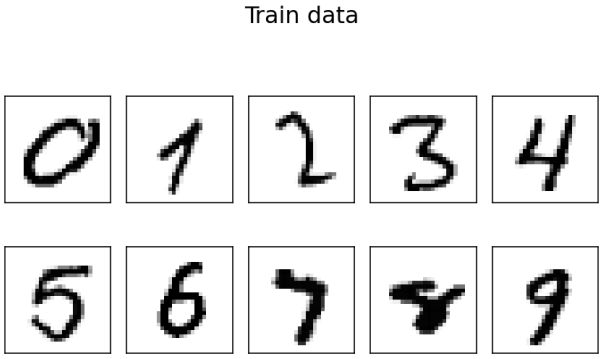


Figure 1: Handwritten number data set used to train the model



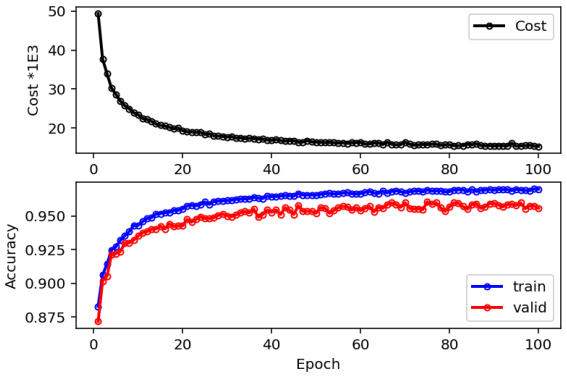
Figure 2: Random handwritten number test data that the algorithm will classify

2 Training the Model

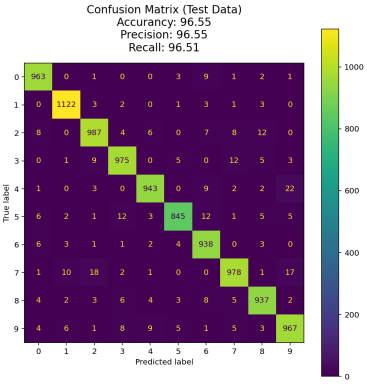
The hyper parameters for training the model are the following. The demission of the image is 28 x 28 pixel, resulting in a 728 matrix which are the nx, ny variables.

```
#--------hyper parameters---------------
n_iter   = 100
d_hidden = 100 # number of hidden units
L2_reg   = 0.1 #factor for regularization term
eta      = 0.001 # learning rate
#----------training and testing sizes---
n_minibatch = 100
valid_size  = 0.05 # check model perfomance during testing
```

After training the model and do the classification I get the following results:



(a) Performance



(b) Confusion matrix performance

Figure 3: Plots that show the performance of the Model after training with 0.10% of the data set, using 100 hidden layers and 100 epochs

```
#Find the absolute most mistaken combination (off-diagonal maximum)
cm_off_diagonal = cm.copy()
np.fill_diagonal(cm_off_diagonal, -1)

# Get the indices of the highest value left over
true_digit, pred_digit = np.unravel_index(np.argmax(cm_off_diagonal), cm_off_diagonal.
    shape)
max_mistakes = cm_off_diagonal[true_digit, pred_digit]

# Find the hardest digit overall (lowest recall / digit accuracy)
class_accuracies = cm.diagonal() / cm.sum(axis=1)
hardest_digit = np.argmin(class_accuracies)
hardest_digit_acc = class_accuracies[hardest_digit] * 100


print(f"Most Mistaken Pairing: True digit '{true_digit}' was mistaken for '{pred_digit}'
    {max_mistakes} times.")
print(f"Lowest Performing Class: Digit '{hardest_digit}' was the hardest to identify.")
```

For answering the question, I use the confusion matrix shown in figure 3, evaluating the behavior of the error in the predictions, which simply consist on find the highest value of the errors on the predicted digit and used as a inconsistency measurement and then find the hardest digit as the ratio of the lowest recall over the summation of the digit accuracy, this what this code does. So that:

- **Which number is most easily mistaken?:** The most easily mistaken number is 6, because it is the second digit with the lowest value in the diagonal and with a high spread error in the predictions.
- **Which number is generally the hardest to predict?:** 5 is the hardest to identify, which got the lowest accuracy (94%) when the average performance is 96%. In fact, the lowest value in the diagonal of the confusion matrix is 5 with a value of 845.
- **Which two number appear to be the most similar:** the pair 4,9 and 7,9 appear to be the most similar pairs number according to the number of mistaken predictions.

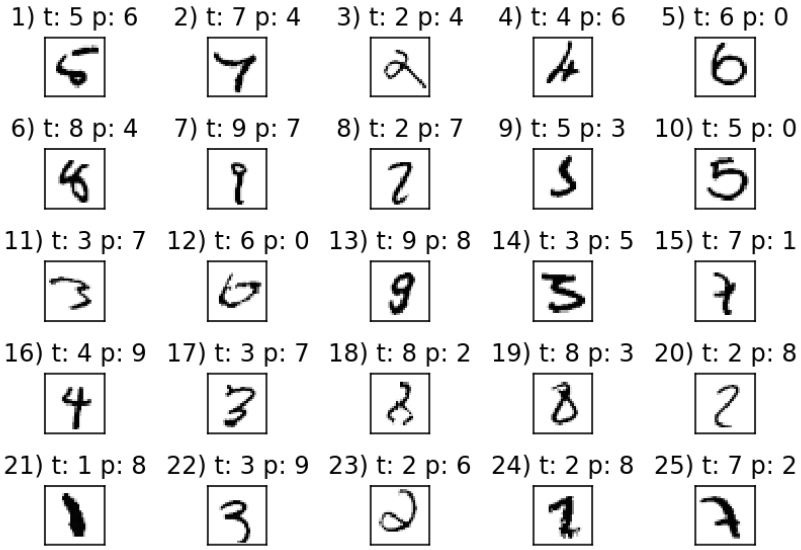


Figure 4: Random miss-classified numbers

3 Further Analysis

3.1 Gris Search

The way to find the hyper-parameters that increase the accuracy to more than 95 percent was done by doing the analysis called grid-search, in which we evaluate the trade-off between two hyper-parameters eg. changing the number of iteration and then evaluate the performance, then change the another hyper-parameter and so on. Doing this until all the selected parameters were tested, and finally find the combination with the best performance. To achieve this task, the hyper-parameters and its values were the following:

- number of iteration: 150 and 200
- hidden layers: 100 and 150
- learning rate: 0.01 and 0.02

The grid-search results are:

```
======================================================================
GRID SEARCH RESULTS
======================================================================
 n_iter  d_hidden   eta  valid_accuracy
    200       150 0.001       96.166667
    200       150 0.002       96.100000
    150       100 0.001       96.033333
    200       100 0.002       96.000000
    150       150 0.002       95.933333
    150       150 0.001       95.900000
    200       100 0.001       95.900000
    150       100 0.002       95.533333

======================================================================
BEST CONFIGURATION
======================================================================
n_iter:     200
d_hidden:   150
eta:        0.001
Validation: 96.17%
======================================================================
```

The final result is:

```
======================================================================
FINAL TEST PERFORMANCE
======================================================================
Accuracy:  96.90%
Precision: 96.90%
Recall:    96.87%
F1-score:  96.88%
Target >95% accuracy: ACHIEVED
======================================================================
```

We can conclude that the initial guest of 100 iterations, 100 hidden layers and 0.001 learning rate, were a good choose, but in order to have more constraint predictions, the best parameters are 200 epoch, 150 hidden layers and 0.001 learning rate.

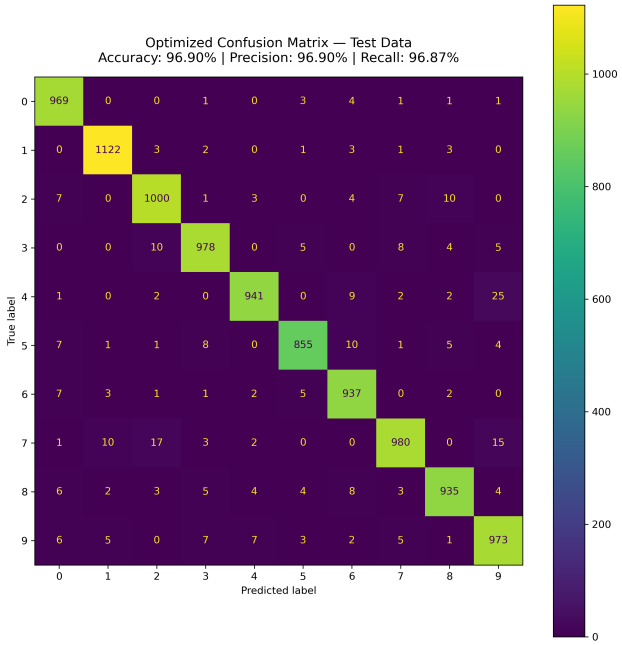


Figure 5: Confusion matrix with the optimize hyper-parameters

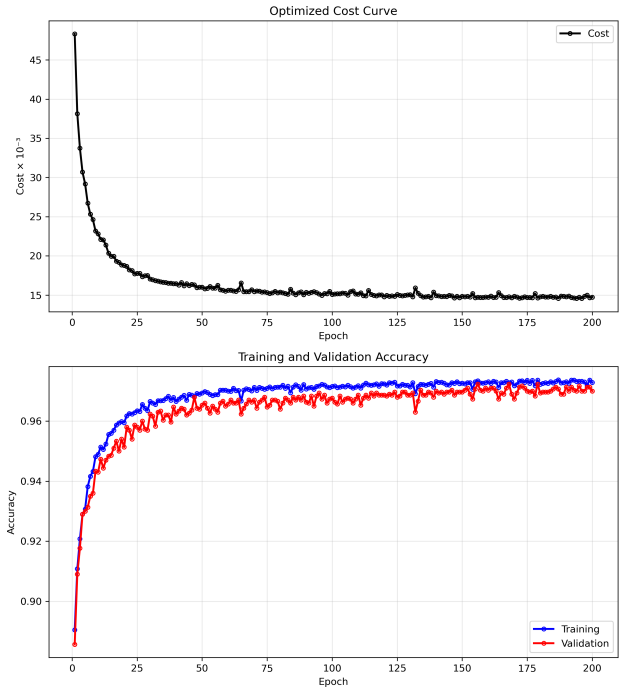


Figure 6: Figure showing the cost and accuracy of the model's behavior using the optimize hyper-parameters values

3.2 trade-offs between training and validation data size.

In this analysis, I trained a neuronal network using a for loop where each iteration varied the training and validation data set sizes by 10%, staring with 95% of the data (10% of the entire catalog) and ending with 5%. At first glance, the training and validation curves in light blue (5%) appear more stable with a straight line, while the dark blue line trained with 95% of the data exhibits the worst performance (see figure 7). However, the cost function accumulates error linearly across the total sample, this means that a larger number of samples could yield a higher cumulative error. For this reason, and to obtain a more constrained analysis, we need to examine the accuracy performance of each iteration. Here, the results are complete different. We are now comparing the model's capacity to predict outcomes; as expected, the less data used in training, the worse its capacity to make an accurate prediction. Ultimately, the model's performance relies heavily on how closely the evaluation data resembles the training dataset, this can be easily see in figure 8 that show the model's metric performance (accuracy,precision and recall), of each iteration as a function of the % of data used, and we can see how fast the performance drops when the data set size decreases.

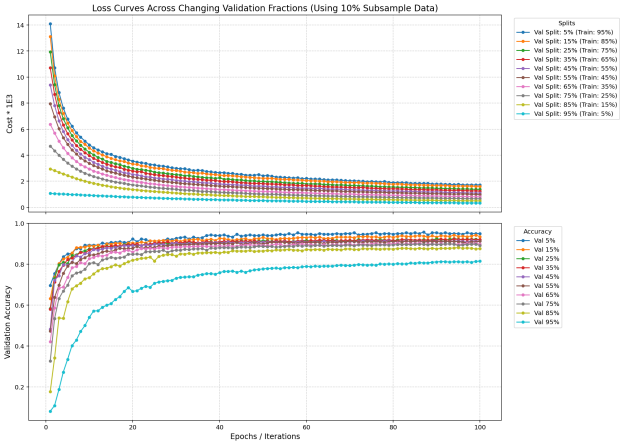


Figure 7: Comparison of the performance of the model using 10% of the data, and changing the train/validation values from 5 to 95 percent in 10% increment.



Figure 8: Caption